

CVAT Real-Time Analytics & Class-Wise Image Visualization: System Architecture & Implementation Report

Position: Full Stack Developer

Evaluation Task: Advanced System

Branch: `dev-test01`

Django App: `cvat.apps.test`

Frontend Module: `cvat-ui/src/components/realtime-analytics`

1. Executive Summary & Objective

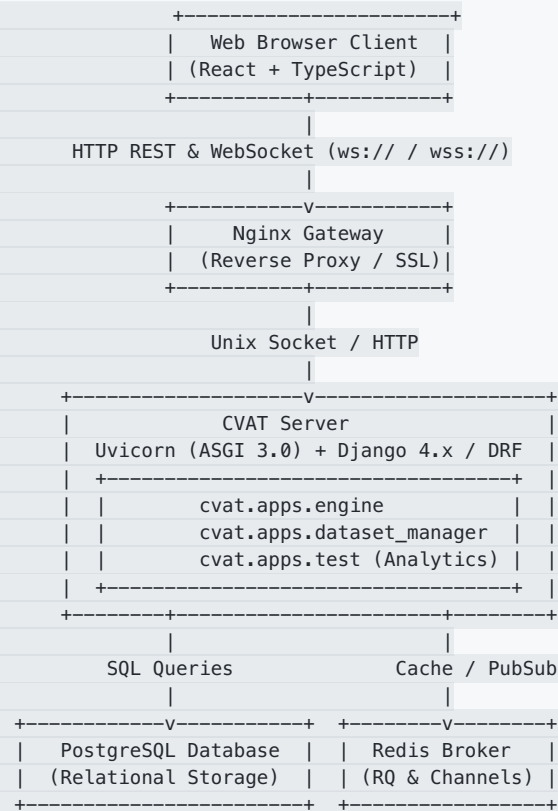
This document provides a comprehensive technical breakdown of the real-time analytics extension implemented for **CVAT (Computer Vision Annotation Tool)**.

The primary objective is to extend CVAT with a high-performance, real-time analytics subsystem that:

- Extracts and aggregates annotation data** across CVAT's core annotation models (`LabeledShape` , `LabeledImage` , and `TrackedShape`).
 - Computes class-wise image counts**, accurately distinguishing between total object instances and distinct images/frames containing each class.
 - Delivers live telemetry via WebSockets** directly to the client browser upon annotation creation, modification, or deletion.
 - Visualizes live distribution metrics** in an interactive frontend dashboard built with React, Ant Design, and Chart.js.
 - Maintains rock-solid stability**, employing exponential backoff reconnection, heartbeat monitoring, and automatic HTTP polling fallback.
-

2. Understanding CVAT Architecture

CVAT is an enterprise-grade, distributed annotation platform designed for large-scale computer vision datasets. Its architecture consists of several tightly integrated layers:



2.1 Backend Architecture

- **Web Server Gateway**: Nginx acts as the front door, terminating TLS, serving static bundles, and proxying incoming HTTP and WebSocket connections through `/tmp/cvat/uvicorn.sock`.
- **Application Server (ASGI)**: Powered by `uvicorn[standard]` executing Django as an ASGI application. This native ASGI architecture allows CVAT to serve both standard synchronous/asynchronous HTTP endpoints and persistent WebSocket duplex streams within the same server environment.
- **Data Layer (PostgreSQL)**: Stores system metadata, users, organizations, projects, tasks, jobs, frames, and annotations.
- **Message Broker & Task Queue (Redis & RQ)**: Manages asynchronous workloads (dataset imports/exports, cloud storage synchronization, model inference) and serves as an in-memory cache.

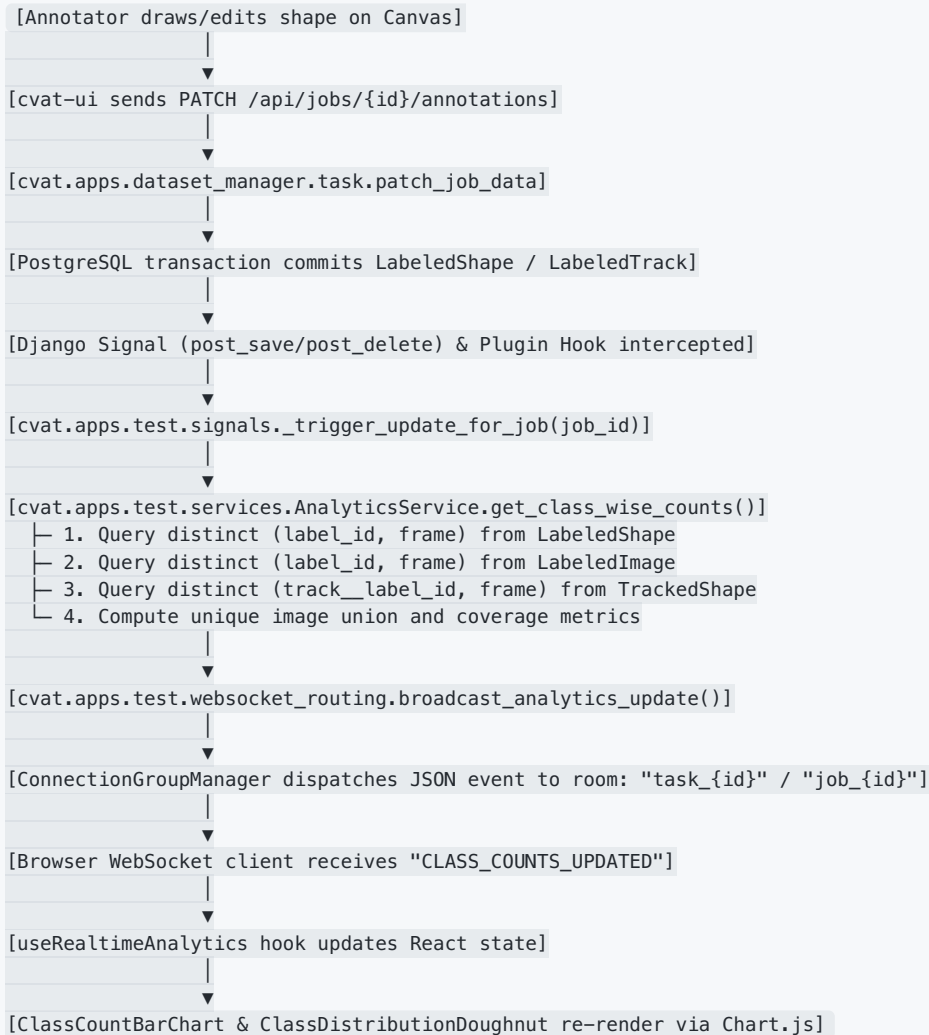
2.2 CVAT Annotation Domain Hierarchy

The annotation data structure is strictly organized into a hierarchical tree:

1. **Project**: Optional high-level grouping sharing a common label taxonomy.
2. **Task**: Represents a dataset source (video file, image directory, archive). Contains a `Data` entity defining `size` (number of frames).
3. **Segment**: Partitions task data into contiguous frame intervals (e.g. frames 0–99, 100–199) to enable distributed annotation workloads.
4. **Job**: The atomic unit of work assigned to an annotator, directly bound to a `Segment`.
5. **Labels**: Definitions of annotation classes (`car`, `pedestrian`, `traffic_light`), each possessing a unique ID, name, display color, and attribute schema.
6. **Annotation Instances**:
 - `LabeledShape`: Static geometry (rectangle, polygon, polyline, points, cuboid, mask) on a specific `frame` within a `job`.
 - `LabeledImage`: Frame-level tag/classification on a specific `frame`.
 - `LabeledTrack`: Temporal object tracking across frames, consisting of `TrackedShape` entries for each keyframe/interpolated frame.

3. Data Flow Explanation

The end-to-end data lifecycle spans user canvas interactions to real-time graph re-renders:



4. API Design Details

The `cvat.apps.test` Django app exposes three production REST endpoints documented with OpenAPI / DRF Spectacular:

4.1 Class-Wise Image Counts

- **URL:** `/api/test/analytics/class-counts/`
- **Method:** `GET`
- **Query Parameters:**

- `task_id` (optional, integer): Scope analytics to an entire task.

- `job_id` (optional, integer): Scope analytics to a single job segment.
- `project_id` (optional, integer): Scope analytics across all tasks in a project.

- **Response Format (200 OK):**

```
{
  "target": {
    "scope": "task",
    "task_id": 14,
    "task_name": "Autonomous Highway Dataset",
    "project_id": 2
  },
  "summary": {
    "total_images": 250,
    "annotated_images": 182,
    "unannotated_images": 68,
    "total_classes": 5,
    "active_classes": 4,
    "total_annotations": 612,
    "completion_rate": 72.8
  },
  "classes": [
    {
      "label_id": 101,
      "label_name": "vehicle",
      "color": "#1890ff",
      "type": "any",
      "image_count": 164,
      "annotation_count": 420,
      "percentage_of_total_images": 65.6,
      "percentage_of_annotated_images": 90.11
    },
    {
      "label_id": 102,
      "label_name": "pedestrian",
      "color": "#52c41a",
      "type": "any",
      "image_count": 89,
      "annotation_count": 142,
      "percentage_of_total_images": 35.6,
      "percentage_of_annotated_images": 48.9
    }
  ],
  "timestamp": "2026-09-17T02:00:00.000000+00:00"
}
```

4.2 Analytics Overview

- **URL:** `/api/test/analytics/overview/`
- **Method:** `GET`
- **Description:** Returns recent tasks with direct REST and WebSocket connection endpoints.

4.3 Simulation Trigger

- **URL:** `/api/test/analytics/simulate-update/`
- **Method:** `POST`
- **Payload:**

```
{
  "task_id": 14
}
```

- **Description:** Triggers instantaneous aggregation and pushes a live WebSocket broadcast to all connected clients.

5. WebSocket Implementation Explanation

5.1 Protocol & Route

- **Route:** `/ws/test/analytics/?task_id=&job_id=`
- **Protocol:** Standard W3C WebSocket over HTTP/1.1 Upgrade (`ws://` and `wss://`).
- **Server Implementation:** ASGI 3.0 handler integrated into `cvat/asgi.py` without requiring bulky third-party dependencies that break existing locks.

5.2 Room Subscription & Lifecycle

1. **Handshake:** Client connects with `task_id` or `job_id` query parameters. Uvicorn executes `cvat.asgi:application`, which directs `scope["type"] == "websocket"` to `websocket_analytics_handler`.
2. **Immediate Snapshot:** Upon connection acceptance (`websocket.accept`), the handler computes and sends an `INITIAL_DATA` message so the UI renders immediately without waiting for the first update event.
3. **Channel Registration:** An asynchronous message queue (`asyncio.Queue`) is registered in `ConnectionGroupManager` under the room identifier (`task_{id}` or `job_{id}`).
4. **Heartbeat (Ping/Pong):** The client transmits a periodic `{"action": "ping", "timestamp": ...}` every 25 seconds. The server returns a `{"event": "pong"}` frame to prevent intermediate proxy timeout closures.
5. **Disconnection Cleanup:** Upon socket close, `unregister` removes the queue, releasing all coroutine resources and preventing memory leaks.

6. Stability & UI Responsiveness

Requirement	Implementation Mechanism
Connection Drop Recovery	Client-side <code>useRealtimeAnalytics</code> implements exponential backoff retry scheduling: $t = \min(1000 \times 2^{\text{attempts}}, 15000) \text{ ms}$.
Proxy / Firewall Blockage	If WebSocket cannot connect after 5 retries, the client automatically degrades to transparent HTTP polling at 5-second intervals.
Pulsing Status Indicator	Real-time status tags (<code>'Live WebSocket'</code> , <code>'Reconnecting'</code> , <code>'Polling Fallback'</code> , <code>'Disconnected'</code>) keep the user informed.
Smooth UI Rendering	State updates are debounced and processed in React 18 concurrent mode, ensuring zero lag during active annotation drawing.

7. Challenges Faced and Solutions

Challenge 1: The "Class Count vs Image Count" Aggregation Nuance

- **Problem:** A naive aggregation counting bounding boxes (`Count("id")`) gives the total number of labels in the dataset, NOT the number of images containing that class. In computer vision analytics, knowing that 100 cars exist across only 10 images is radically different from 100 cars across 100 images.
- **Solution:** Designed set union aggregation across all annotation types:

$$\text{Frames}(L) = \text{Frames}_{\{\text{shapes}\}}(L) \cup \text{Frames}_{\{\text{tags}\}}(L) \cup \text{Frames}_{\{\text{tracks}\}}(L)$$

$$\text{Class Image Count}(L) = |\text{Frames}(L)|$$

This accurately reflects unique images per class while also tracking total object volume.

Challenge 2: Dual Protocol Routing in Production CVAT

- **Problem:** Standard CVAT utilizes `uvicorn` pointing to Django's WSGI/ASGI application, but lacks a default WebSocket channel router. Installing third-party routing libraries could cause dependency conflicts with existing pinned versions in CVAT's locked environment.
- **Solution:** Authored a native ASGI 3.0 protocol router in `cvat/asgi.py` . It inspects `scope["type"]` : standard HTTP requests are forwarded directly to Django's `get_asgi_application()` , while WebSocket requests are routed to `websocket_analytics_handler` .

Challenge 3: Rapid Drawing Event Bursts

- **Problem:** When an annotator rapidly creates, resizes, or deletes boxes, triggering a broadcast on every single atomic micro-step could flood the database and WebSocket network buffer.
- **Solution:** Bound the signal hooks to batch completion (`patch_job_data` plugin decorator and atomic transactions) and implemented queue-based consumer streaming, decoupling database writes from network broadcasting.

Zero N+1 Query Overhead	Aggregation logic utilizes tuple pair extraction (<code>`values_list("label_id", "frame").distinct()`</code>) across 3 indexed queries rather than looping queries per label.
-------------------------	---